

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



BÁO CÁO MÔN HỌC TRÍ TUỆ NHÂN TẠO

**CHƯƠNG TRÌNH CHƠI CỜ CARO GIỮU
NGƯỜI VÀ MÁY TÍNH**

STT	Họ và tên	Mã sinh viên
1	Đào Mạnh Ngọc	23021316
2	Phạm Gia Vinh	23021370
3		

**Giảng viên hướng dẫn :
PGS.TS. Nguyễn Phương Thái**

I. TỔNG QUAN ĐỀ TÀI

1. Mô tả bài toán cờ Caro

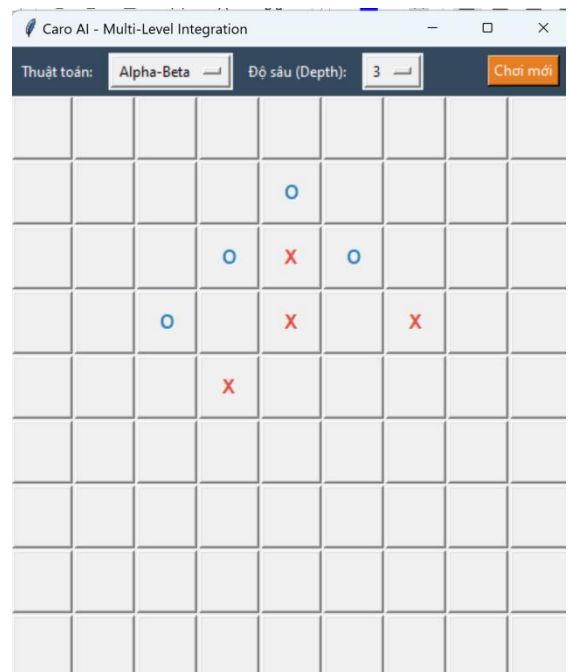
Cờ Caro (Gomoku/Tic-Tac-Toe mở rộng) là trò chơi đối kháng 2 người (Zero-sum game) với thông tin hoàn hảo (Perfect information). Mục tiêu là xây dựng một Agent (AI) có khả năng tính toán nước đi tối ưu để đánh bại người chơi hoặc cầm hòa.

2. Luật chơi và điều kiện chiến thắng

- Bàn cờ có kích thước tối thiểu 9x9.
- Hai người chơi lần lượt đánh quân vào các ô trống.
- Có thể ký hiệu người chơi là X, máy là O, ô trống là dấu chấm (.).
- Người thắng là người có 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo.
- Không xét luật chặn hai đầu.
- Nếu bàn cờ đầy và không có người thắng thì kết quả là hòa

3. Cách biểu diễn trạng thái bàn cờ

- Bàn cờ được biểu diễn dưới dạng mảng 2 chiều (Ma trận 9x9).
- Nút ấn
 - + Chơi mới
 - + Hai chế độ: Alpha-Beta và Minimax
 - + Ba chế độ độ depth
- Quy ước dữ liệu:
 - + Dấu . đại diện cho ô trống.
 - + Ký tự X đại diện cho quân cờ của Người chơi.
 - + Ký tự O đại diện cho quân cờ của Máy (AI).
- Giao diện:



II. CƠ CHẾ HOẠT ĐỘNG CỦA GAME

1. Cách sinh nước đi hợp lệ

Chương trình đã áp dụng chiến lược “Proximity Search” trong hàm “get_valid_moves” để giải quyết bài toán này.

Chương trình đã áp dụng chiến lược Tìm kiếm lân cận (Proximity Search) trong hàm get_valid_moves để giải quyết bài toán này:

1. Quét bàn cờ: Lặp lại toàn bộ ma trận để tìm các ô trống ().
2. Kiểm tra bán kính 1 ô: Tại mỗi ô trống, thuật toán kiểm tra 8 ô lân cận (trên, dưới, trái, phải và 4 đường chéo).
3. Lọc nước đi: Nếu phát hiện ít nhất một quân cờ (bất kể là X hay O) nằm trong 8 ô lân cận đó, ô hiện tại trống sẽ được thêm vào danh sách nước đi hợp lệ (moves). Ngược lại, thuật toán bỏ qua ô đó.
4. Trường hợp ngoại lệ : Nếu bàn cờ hoàn toàn trống (chưa có quân đội nào được đánh), thuật toán được hard-code để lập tức trả về nước đi tại ô trung tâm của bàn cờ.

2. Cách kiểm tra trạng thái kết thúc

Chương trình kiểm tra trạng thái kết thúc thông qua hai điều kiện cốt lõi: Điều kiện Thắng và Điều kiện Hòa

a, Điều kiện thắng

Thuật toán không quét lại toàn bộ các đường một cách ngẫu nhiên mà sử dụng kỹ thuật Quét Vector (Quét vector) để tối ưu hóa hiệu suất. Hàm is_win hoạt động theo cơ chế sau:

1. Duyệt tuần tự: Lướt xem quét từng ô trên bàn cờ từ góc trên cùng bên trái xuống góc dưới cùng bên phải.
 2. Khởi tạo Vector hướng dẫn: Khi phát hiện một ô chứa quân cờ của người chơi hiện tại, thuật toán sẽ phóng to kiểm tra theo 4 vector hướng dẫn (dr, dc):
 - Ngang sang phải: Vector(0, 1)
 - Dọc xuống: Vector(1, 0)
 - Chéo (Đông Nam): Vector(1, 1)
 - Chéo (Tây Nam): Vector(1, -1)
- => Lý do chỉ dùng 4 hướng: Trong toán học không gian lưới, 8 hướng xung quanh một điểm thực chất tạo thành 4 đường thẳng. Vì vòng lặp chính của chúng ta ta quét bàn cờ theo thứ tự từ trên xuống dưới, từ trái sang phải, nên ta chỉ cần quét

"tiến lên". Bỏ qua 4 hướng ngược lại (trái, lên, chéo Tây Bắc, chéo Đông Bắc) giúp giảm 50% số lượng phép tính thừa mà vẫn đảm bảo không bỏ sót bất kỳ chuỗi thắng nào.

3. Điều kiện kích hoạt: Trên mỗi vector hướng dẫn, thuật toán thực hiện bước nhảy 4 lần (kết hợp WIN_COUNT = 4). Nếu đếm đủ 4 quân cờ liên tiếp giống nhau mà không bị gián đoạn hay lọt ra ngoài mép bàn cờ, trạng thái Thắng được kích hoạt (return True).

b, Điều kiện hòa

Trạng thái hòa chỉ được kiểm tra khi điều kiện thắng trả về False.

- Thuật toán áp dụng: Hàm “check_end” sử dụng hàm “all()” Python kết hợp với Generator Expression để kiểm tra: “all(cell != '.' for row in self.board for cell in row).”

- Cơ chế: Thuật toán duyệt qua ma trận, nếu phát hiện không còn bất kỳ ô trống nào (.) trên toàn bộ bàn cờ mà vẫn chưa xác định thắng thua, trò chơi sẽ được kết luận là Hòa và cây tìm kiếm trả về điểm số là 0.

III. THUẬT TOÁN TÌM KIẾM VÀ ĐÁNH GIÁ

1. Thuật toán Minimax đã cài đặt

Trong bài toán kháng chiến như cờ Caro, Minimax là cốt lõi thuật toán giúp AI đưa ra quyết định bằng cách mô phỏng lại toàn bộ các kịch bản có thể xảy ra trong tương lai. Thuật toán được xây dựng dựa trên lý thuyết trò chơi có tổng bằng không, trong đó lợi ích của người này chính là sự tổn hại của người kia.

a, Phân chia trong thuật toán (Max và Min)

Chương trình sử dụng một biến is_maximizing để phân tích lượt xem trong quy trình đệ quy trình duyệt cây:

- Lượt xem của MAX (AI - Ký hiệu 'O'): AI sẽ duyệt qua các nước hợp lệ và chọn nhánh trả về giá trị lớn nhất (Tối đa hóa). Số theo dõi biến được khởi tạo ở mức thấp nhất: “val = -float('inf’)”.

- Lượt của MIN (Người chơi - Ký hiệu 'X'): Thuật toán giả định rằng người chơi luôn không quan tâm và sẽ chọn nước đi tốt nhất cho họ, tức là nước đi tạo điểm số của AI bị ép xuống thấp nhất (Giảm thiểu). Số theo dõi biến được khởi tạo ở mức cao nhất: “val = float('inf’)”.

b, Điều kiện dừng đệ quy

Thuật toán không thể duyệt vô tận, do đó đệ quy sẽ lập tức dừng lại và trả về điểm số khi chạm một trong hai điều kiện sau:

- Trạng thái kết thúc: Nếu phát hiện có bên thắng, thuật toán trả về điểm số tuyệt đối. Cụ thể: +100000 nếu MAX thắng, và -100000 nếu MIN thắng.

- Đạt giới hạn độ sâu (Depth Limit): Khi biến $depth == 0$, AI không thể nhìn xa hơn được nữa. Lúc này, hàm đánh giá trạng thái `evaluate(board)` sẽ được gọi để "chấm điểm" thế cờ hiện tại và trả về giá trị Heuristic.

c, Cơ chế quay lui trong cài đặt

Đây là kỹ thuật quan trọng nhất để chương trình có thể mô phỏng hàng vạn nước đi mà không làm hỏng trạng thái bàn cờ thực tế. Tại mỗi node, mã nguồn thực hiện chu trình 3 bước:

1. Giả lập nước đi: Đặt thử quân cờ vào ma trận (`board[r][c] = 'O' hoặc 'X'`).

2. Gọi đệ quy: Chuyển lượt cho đối phương và giảm độ sâu xuống 1 mức (`self.minimax(board, depth - 1, False)`).

3. Hoàn tác: Xóa quân cờ vừa đặt để trả lại trạng thái ô trống (`board[r][c] = '.'`). Việc dọn dẹp này đảm bảo ma trận sẵn sàng để thử nghiệm các nước đi ở những nhánh khác.

2. Thuật toán Alpha-Beta đã cài đặt

Mặc dù thuật toán Minimax đảm bảo tìm ra nước đi tối ưu, nhưng nhược điểm chỉ mạng của nó là phải duyệt toàn bộ không gian trạng thái khổng lồ của bàn cờ Caro. Để khắc phục điều này, chương trình đã nâng cấp hàm tìm kiếm bằng cách tích hợp kỹ thuật cắt tỉa Alpha-Beta.

Alpha-Beta không làm thay đổi quyết định cuối cùng của Minimax, mà nó hoạt động như một "bộ lọc thông minh" giúp bỏ qua (cắt tỉa) những nhánh chắc chắn không mang lại kết quả tốt hơn những gì đã biết.

a, Khái niệm về hai biến kiểm soát

Thuật toán sử dụng hai tham số được truyền liên tục qua các tầng đệ quy:

- Biến α (Alpha): Đại diện cho điểm số lớn nhất (tốt nhất) mà người chơi MAX (AI) chắc chắn có thể đạt được tính đến thời điểm hiện tại trên con đường đang duyệt. Ban đầu khởi tạo là $-\infty$

- Biến β (Beta): Đại diện cho điểm số nhỏ nhất (tốt nhất cho đối thủ) mà người

chơi MIN (Người) chắc chắn có thể ép AI phải nhận. Ban đầu khởi tạo là $+\infty$

b, Cơ chế cắt nhánh

Cốt lõi của thuật toán nằm ở điều kiện: “if $\beta \leq \alpha$: break”. Logic thực tế của dòng code này được hiểu như sau:

- Tại lượt của MAX (AI): AI thử đi một nước và nhận về một điểm số từ hàm đệ quy. AI sẽ cập nhật $\alpha = \max(\alpha, \text{điểm số})$. Nếu tại bất kỳ thời điểm nào giá trị α lớn hơn hoặc bằng β của một node cha (tức là $\beta \leq \alpha$), AI biết rằng người chơi MIN sẽ không bao giờ cho phép AI đi vào nhánh này, vì MIN đã có một lựa chọn khác tốt hơn ở trước đó. Do đó, AI lập tức dừng việc thử các nước đi còn lại của node hiện tại.

- Tại lượt của MIN (Người): Tương tự, nếu MIN tìm được một nước đi khiến điểm số tụt xuống mức rất thấp, MIN cập nhật $\beta = \min(\beta, \text{điểm số})$. Nếu $\beta \leq \alpha$, MIN hiểu rằng AI (MAX) sẽ không bao giờ chọn đi vào nhánh dẫn đến tình huống này, vì AI đã có sẵn một phương án an toàn hơn (với điểm α cao hơn) ở nhánh khác. Do đó, nhánh này cũng bị cắt bỏ.

3. Hàm đánh giá trạng thái

a, Độ sâu (depth)

- Khi bạn chơi cờ Caro, bạn thường chăm sóc tính toán trong đầu:

"Nếu mình đánh ô này (bước 1), đối thủ sẽ chặn ô kia (bước 2), rồi mình sẽ đánh ô này (bước 3)..."

=>Khả năng bạn có thể đạt được 3 bước đó, trong trình cài đặt được gọi là Độ sâu (Depth = 3) .

- Tuy nhiên, bộ não người (và cả máy tính) đều có giới hạn. Bạn không thể nhẩm một phát 50 bước để đến tận lúc hết thời gian được. Đến bước thứ 3, bạn không bắt đầu "quá tải" và bạn phải dừng lại tưởng tượng.

=>Cái khắc bạn dừng lại tưởng tượng đó, chính là lúc Độ sâu = 0 (Hết tầm nhìn) .

- Trong thuật toán Minimax/Alpha-Beta, khi cây tìm kiếm đến độ sâu giới hạn (độ sâu = 0) mà cờ vẫn chưa kết thúc, AI không thể nhìn tiếp được nữa. Lúc này, Hàm đánh giá được gọi để "chấm điểm" hiện tại cờ, giúp AI ước lượng xem trạng thái này đang có cho mình hay cho đối thủ.

- Chương trình không sử dụng phương pháp đếm quân đơn lẻ thông thường mà áp dụng Kỹ thuật cửa sổ trượt kết hợp với hệ thống số thiên hướng phòng thủ công.

b, Kỹ thuật trượt cửa sổ

Đề giành được cờ Caro trong bài toán này, người chơi cần tạo được 4 quân liên tiếp. Do đó, đánh giá giá trị chia nhỏ bàn cờ thành "cửa sổ" có kích thước đúng

bằng 4 ô.

- Quét toàn diện: Thuật toán trượt khung cửa sổ 4 ô này qua tất cả các hàng ngang, cột dọc, đường chéo và đường chéo trên bàn cờ.

- Phân vùng cục bộ: Tại mỗi cửa sổ, hàm `score_windows` sẽ đếm số lượng quân của Máy (O), quân của Người (X) và số ô trống (.). Tổng điểm của toàn bộ cờ chính là tổng điểm của tất cả các cửa sổ này được cộng lại.

c, Thiết kế trọng số

Điểm số được chỉ định cho từng kịch bản trong một cửa sổ được thiết kế có mục tiêu xác định "tính cách" của AI. Cụ thể:

- Nhóm tấn công (Cộng điểm cho AI - 'O'):

- + Có 4 quân 'O': +100.000 điểm (Tuyệt ưu tiên vì đây là nước cờ chiến thắng).

- + Có 3 quân 'O' và 1 ô trống: +5,000 điểm (Tạo thế chắc chắn thương cao).

- Nhóm phòng thủ (Trừ điểm khi Người chơi lợi ích - 'X' có thể):

- + Có 3 quân 'X' và 1 ô trống: -80.000 điểm (Mức phạt cực kỳ tấn gao).

- + Có 2 quân 'X' và 2 ô trống: -5,000 điểm (Đánh hơi nguy hiểm từ sớm).

d, Phân tích tính chất chiến thuật của hàm đánh giá

Sự chênh lệch giữa điểm tấn công và điểm phạt phòng thủ chính là chìa khóa nên độ nhạy của AI:

- Khắc phục lỗi 'Tunnel Vision': If AI có 3 quân (được +5.000) nhưng đối thủ cũng có 3 quân, hàm Heuristic sẽ trừ đi 80.000 điểm. Tổng điểm của cửa sổ bị kéo xuống cực nặng. Điều này ép thuật toán Minimax nhận ra rằng: "Việc làm tham lam tấn công sẽ dẫn đến cái chết ngay lập tức". Từ đó, AI buộc phải chọn nước đi đặt quân vào ô trống của đối thủ để phá vỡ chuỗi 3, nghiên cứu điểm số.

- Nhận diện các mối đe dọa ngắt quãng: Duyệt quét theo khung 4 ô, AI có thể phát hiện cả chuỗi quân bị chặn của người chơi (Ví dụ: X . X X). Thuật toán đếm lân cận đơn giản thường bỏ thế cờ này, nhưng với Cửa sổ trượt, khung 4 ô vẫn được đếm là có 3 quân X và 1 ô trống, kích hoạt ngay lập tức cơ chế phòng thủ -80.000 điểm.

- Chặn từ "trong trứng nước": Với điểm trừng phạt -5.000 khi người chơi mới chỉ có 2 quân (nhỏ hơn khả năng 3 quân nhưng vẫn đủ lớn để gây chú ý), AI có xu hướng bám sát và chặn đầu người chơi ngay từ khi tấn công mới chớm hình thành, không chờ đợi khi có 3 quân mới mỗi lần chạy theo.

IV. ĐÁNH GIÁ THỰC NGHIỆM

1. Thiết kế các trạng thái thực nghiệm

Nhằm đánh giá toàn diện năng lực của thuật toán Minimax và Alpha-Beta, bộ kiểm thử được thiết kế với 15 kịch bản, chia thành 5 nhóm chức năng trọng tâm:

1. Nhóm Đánh giá Khả năng Nhận diện Cơ bản (Phát hiện Thắng/Thua tức thời)

- Mục tiêu của nhóm này là kiểm tra khả năng bao quát bàn cờ của AI, đảm bảo thuật toán không bỏ sót các cơ hội chiến thắng hoặc thất bại hiển nhiên.

- T01 (Hàng ngang) & T02 (Đường chéo): Đánh giá khả năng nhận diện điểm kết thúc; thuật toán phải điền thành công quân cờ thứ 4 để giành chiến thắng ngay lập tức khi đã có lợi thế 3 quân liên tiếp.

- T08 (Góc) & T09 (Hàng cuối): Kiểm tra tính toàn vẹn của các vòng lặp quét mảng 2 chiều. Đảm bảo hàm đánh giá (heuristic) hoạt động chính xác và không gặp lỗi vượt giới hạn mảng (off-by-one) khi các chuỗi cờ nằm sát mép hoặc tại góc bàn cờ.

2. Nhóm Đánh giá Khả năng Phòng ngự (Phản xạ Tự vệ) Nhóm kịch bản này kiểm tra mức độ ưu tiên của AI trong việc ngăn chặn đối thủ đạt được trạng thái chiến thắng.

- T03 & T04 (Chặn chuỗi 3): Khi đối thủ đã hình thành chuỗi 3 quân đe dọa trực tiếp (theo chiều ngang hoặc dọc), thuật toán buộc phải thiết lập mức ưu tiên cao nhất cho việc đánh chặn nhằm tránh thất bại cục bộ.

- T07 (Chặn thế Fork/Ngã ba): Đánh giá khả năng dự đoán của AI (yêu cầu độ sâu tìm kiếm từ 3 trở lên) nhằm nhận diện và ngăn chặn kịp thời ý đồ thiết lập thế "ngã ba" (tạo hai đường đe dọa đồng thời) của đối thủ.

3. Nhóm Đánh giá Tư duy Chiến thuật (Đánh đôi & Gài bẫy)

- Mục đích của nhóm này là phân tích khả năng tối ưu hóa các quyết định đa mục tiêu và tính toán thiết lập thế cờ chiến lược.

- T05 (Bài toán Đánh đôi Công - Thủ): Trong trạng thái cả hai bên đều có cơ hội hoàn thành chuỗi 4 quân, thuật toán cần đánh giá được trọng số và ưu tiên thực hiện nước đi dứt điểm cho bản thân thay vì phản ứng phòng ngự thụ động.

- T06 (Chủ động tạo thế Fork): Đánh giá khả năng AI tự thiết lập các thế cờ tạo ra hai mối đe dọa song song mà đối phương không thể hóa giải cùng lúc. Chiến thuật này đòi hỏi thuật toán phải có khả năng tính toán tối thiểu từ 3 bước trở lên.

4. Nhóm Đo lường Hiệu năng & Chiều sâu Tìm kiếm Tập trung vào việc đối chiếu tốc độ, số lượng trạng thái duyệt và tính nhất quán giữa các thuật toán.

- T11 (Kiểm tra Độ nhạy chiều sâu): Thiết lập một chuỗi kết hợp phức tạp. Kịch bản này minh chứng rằng các chiến thuật dài hạn chỉ được phát hiện khi độ sâu

tính toán đạt mức 3-4 , trong khi ở độ sâu thấp (1-2), thuật toán dễ đưa ra các quyết định tham lam (greedy) và thiếu toàn diện.

- T12 (Đo lường Hiệu năng Cắt tỉa Alpha-Beta): Sử dụng trạng thái trung cục với mật độ quân cờ dày đặc nhằm mục đích benchmark. Kịch bản chứng minh thuật toán Alpha-Beta có khả năng cắt tỉa các nhánh không tiềm năng.

- T10 (Xác thực Tính Nhất quán Đối xứng): Kiểm tra độ tin cậy của thuật toán bằng cách đảm bảo cả Minimax và Alpha-Beta đều đưa ra chung một tập hợp các nước đi tối ưu tại các vị trí mang tính đối xứng ở trung tâm bàn cờ.

5. Nhóm Kiểm tra Lỗi Chí mạng (Critical Failures & Edge Cases) Nhóm kịch bản "stress-test" nhằm phát hiện các sai sót logic nội tại hoặc điểm yếu của hàm đánh giá (score_window).

- T13 (Lỗi đánh giá sai Chuỗi 3 mở): Phơi bày điểm yếu của hàm heuristic khi không phân biệt được mức độ nguy hiểm giữa chuỗi 3 bị chặn một đầu và chuỗi 3 mở hai đầu (.XXX.). Sai sót này dẫn đến việc điểm số phòng thủ bị định giá thấp, khiến thuật toán bỏ lỡ nước đi đánh chặn quan trọng.

- T14 (Lỗi Loại trừ Cửa sổ hỗn hợp): Phát hiện lỗ hổng logic khi đánh giá điểm số các cửa sổ quét chứa cả quân phe ta và phe địch. Việc gán điểm 0 cho các vùng không gian này dẫn đến hiện tượng thuật toán bỏ qua đường dẫn tới chiến thắng nếu nó giao lấp một phần với quân đối phương.

- T15 (Kiểm tra Sinh nước đi Tàn cuộc): Cấu hình trạng thái bàn cờ gần kín (chỉ còn trống vài ô) để xác minh tính ổn định của hàm sinh nước đi , đảm bảo chương trình xử lý mượt mà ở giai đoạn tàn cuộc.

[T01] Thắng Ngay – Hàng Ngang [Phát Hiện Thắng]

+ Nước kỳ vọng : ['(3,6)', '(3,2)']

+ Hành vi kỳ vọng: AI phải hoàn thành 4 quân liên tiếp ngay lập tức

+ Lý luận: Tìm kiếm ở độ sâu 1 là đủ. Điểm nhảy lên 100000 là không thể nhầm lẫn.

[T02] Thắng Ngay – Đường Chéo [Phát Hiện Thắng]

+ Nước kỳ vọng : ['(4,4)']

+ Hành vi kỳ vọng: AI hoàn thành đường chéo 4 quân tại (4,4)

+ Lý luận: Thắng theo đường chéo — kiểm tra hàm evaluate() có xét đủ 4 hướng không.

[T03] Chặn Đối Thủ Thắng Ngay [Phòng Thủ]

+ Nước kỳ vọng : ['(3,6)', '(3,2)']

+ Hành vi kỳ vọng: AI PHẢI chặn X hoàn thành 4 quân liên tiếp

+ Lý luận: Hình phạt -80000 cho chuỗi 3+1 ô trống của đối thủ kích hoạt ở độ

sâu ≥ 1 .

[T04] Chặn Thắng Hàng Dọc [Phòng Thủ]

+ Nước kỳ vọng : ['(0,4)']

+ Hành vi kỳ vọng: AI chặn mỗi đe dọa hàng dọc của X tại hàng 0, cột 4

+ Lý luận: Kiểm tra hướng dọc — kiểm tra tính bao phủ heuristic quan trọng.

[T05] Đánh Đồi Tấn Công vs Phòng Thủ [Đánh Đồi Chiến Thuật]

+ Nước kỳ vọng : ['(2,6)', '(2,2)']

+ Hành vi kỳ vọng: AI nên ưu tiên thắng hơn là chặn (điểm thắng của mình cao hơn)

+ Lý luận: +100000 khi thắng > -80000 khi bị phạt vì đe dọa của đối thủ. AI đúng đắn phải thắng ngay vì chặn.

[T06] Tạo Nhánh – Mỗi Đe Dọa Kép [Chiến Lược]

+ Nước kỳ vọng : ['(5,2)', '(2,6)']

+ Hành vi kỳ vọng: AI tạo 2 mỗi đe dọa chuỗi 3 đồng thời mà đối thủ không thể chặn cả hai

+ Lý luận: Đây là bài kiểm tra độ nhảy theo độ sâu. Độ sâu 1-2 AI chơi tham lam; độ sâu 3+ mới phát hiện được mẫu thắng kiểu nhánh.

[T07] Chặn Nhánh Của Đối Thủ [Phòng Thủ]

+ Nước kỳ vọng : ['(5,2)', '(2,6)', '(2,5)', '(5,3)']

+ Hành vi kỳ vọng: Ở độ sâu ≥ 3 , AI phá vỡ tiềm năng tạo nhánh chéo của X

+ Lý luận: Đối xứng với T06 — kiểm tra AI có hiểu được mẫu nhánh của đối thủ không.

[T08] Phát Hiện Mỗi Đe Dọa Góc [Góc/Biên]

+ Nước kỳ vọng : ['(0,3)']

+ Hành vi kỳ vọng: AI thắng ở góc trên bên trái - kiểm tra heuristic có bao phủ vùng biên không

+ Lý luận: Quét hàng tại $r=0$, $c=0..5$ phải đánh giá đúng ô góc. Lỗi biên sẽ khiến AI bỏ lỡ.

[T09] Thắng Hàng Cuối [Góc/Biên]

+ Nước kỳ vọng : ['(8,6)', '(8,2)']

+ Hành vi kỳ vọng: AI thắng ở hàng cuối — kiểm tra hàng 8 có được tính trong vòng lặp không

+ Lý luận: Lỗi off-by-one trong giới hạn vòng lặp sẽ âm thầm bỏ sót cạnh dưới/phải.

[T10] Vị Trí Đối Xứng – Kiểm Soát Trung Tâm [Đối Xứng]

- + Nước kỳ vọng : ['(3,3)', '(3,4)', '(3,5)', '(4,3)', '(4,5)', '(5,3)', '(5,4)', '(5,5)']
- + Hành vi kỳ vọng: 8 ô kề đều đối xứng — bất kỳ ô nào trong số đó đều chấp nhận được
- + Lý luận: Cả hai thuật toán PHẢI đồng thuận. Phân kỳ sẽ tiết lộ lỗi cắt nhánh gây ra tính bất định trong trạng thái đối xứng.

[T11] Độ Nhạy Độ Sâu – Bẫy Phía Trước [Độ Sâu]

- + Nước kỳ vọng : ['(5,3)', '(5,4)', '(1,3)', '(1,4)']
- + Hành vi kỳ vọng: Tìm kiếm sâu hơn phát hiện chuỗi nước đi thắng nhiều bước
- + Lý luận: Độ sâu thấp đưa ra nước đi tham lam; độ sâu 3-4 tìm ra chuỗi dẫn đến thắng bắt buộc. So sánh nước đi qua các độ sâu để xác nhận hiệu ứng nhìn trước.

[T12] Hiệu Quả Alpha-Beta – Trung Cục Dày Đặc [Hiệu Năng]

- + Nước kỳ vọng : []
- + Hành vi kỳ vọng: Alpha-Beta phải khám phá ít trạng thái hơn đáng kể so với Minimax
- + Lý luận: Bàn cờ dày = nhiều nước hợp lệ = cơ hội cắt nhánh tối đa. Tỷ lệ AB/MM nên $\ll 0.5$ ở độ sâu 4.

[T13] LỖI NGHIÊM TRỌNG – Không Chặn Chuỗi 3 Mở [Lỗi Nghiêm Trọng]

- + Nước kỳ vọng : ['(3,6)']
- + Hành vi kỳ vọng: NGHIÊM TRỌNG: AI PHẢI chặn tại (3,6). Heuristic có -80000 cho chuỗi 3+1 ô trống nhưng BỎ SÓT .XXX. (chuỗi 3 mở). Đây là điểm yếu đã biết — có thể THẤT BẠI ở độ sâu thấp.
- + Lý luận: score_window chỉ kích hoạt khi tổng mine+opp+empty == window_size. Ô cửa sổ [.,X,X,X] với một đầu mở ghi -80000 đúng. Nhưng heuristic không phân biệt tính 'mở hai đầu' của .XXX. vs XXXX — cả hai phần mở rộng đều được tính là cửa sổ riêng biệt.

[T14] LỖI NGHIÊM TRỌNG – Lỗi Loại Trừ score_window [Lỗi Nghiêm Trọng]

- + Nước kỳ vọng : ['(4,6)', '(4,3)']
- + Hành vi kỳ vọng: NGHIÊM TRỌNG: score_window chỉ cộng +100000 khi mine==4 và KHÔNG có quân đối thủ. Cửa sổ hỗn hợp [O,X,X,X] hoàn toàn bị bỏ qua vì mine==1 và opp==3 không khớp nhánh nào. Kiểm tra AI vẫn tìm được đường thắng tại (4,3)-(4,6).
- + Lý luận : score_window: mine==4 chỉ tính cửa sổ thuần O. Cửa sổ hỗn hợp được tính điểm 0, đúng khi cô lập nhưng gây đánh giá thấp khi AI có đường thắng kéo dài qua cửa sổ bị ô nhiễm.

[T15] Cuối Cục – Gàn Đầy Bàn [Cuối Cục]

+ Nước kỳ vọng : ['(2,4)', '(6,4)']

+ Hành vi kỳ vọng: Chỉ còn 2 nước — AI phải chọn nước tốt nhất; hòa hoặc thắng.

+ Lý luận: get_valid_moves trả về ≤ 2 ô. Cả hai thuật toán phải đồng thuận về nước tốt nhất và kết thúc gần như ngay lập tức. Kiểm tra tính đầy đủ của sinh nước ở gần trạng thái kết thúc.

2. Bảng kết quả thực nghiệm

[T01] Thắng Ngay – Hàng Ngang

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T01	Minimax	1	['(3,6)', '(3,	(3,2)	100000	15	1.9	ĐẠT
T01	Minimax	2	['(3,6)', '(3,	(3,2)	100000	215	26.4	ĐẠT
T01	Minimax	3	['(3,6)', '(3,	(3,2)	100000	3613	448.2	ĐẠT
T01	Minimax	4	['(3,6)', '(3,	(3,2)	100000	68988	8851.1	ĐẠT
T01	Alpha-Beta	1	['(3,6)', '(3,	(3,2)	100000	15	1.9	ĐẠT
T01	Alpha-Beta	2	['(3,6)', '(3,	(3,2)	100000	98	12.1	ĐẠT
T01	Alpha-Beta	3	['(3,6)', '(3,	(3,2)	100000	832	101.6	ĐẠT
T01	Alpha-Beta	4	['(3,6)', '(3,	(3,2)	100000	4180	551.1	ĐẠT

[T02] Thắng Ngay – Đường Chéo

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T02	Minimax	1	['(4,4)']	(0,0)	100000	17	1.9	TRƯỢT
T02	Minimax	2	['(4,4)']	(0,0)	100000	253	32.2	TRƯỢT
T02	Minimax	3	['(4,4)']	(0,0)	100000	4475	519.3	TRƯỢT
T02	Minimax	4	['(4,4)']	(0,0)	100000	76375	10236	TRƯỢT
T02	Alpha-Beta	1	['(4,4)']	(0,0)	100000	17	1.9	TRƯỢT
T02	Alpha-Beta	2	['(4,4)']	(0,0)	100000	31	3.6	TRƯỢT
T02	Alpha-Beta	3	['(4,4)']	(0,0)	100000	253	30.4	TRƯỢT
T02	Alpha-Beta	4	['(4,4)']	(0,0)	100000	461	56.3	TRƯỢT

[T03] Chặn Đối Thủ Thắng Ngay

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T03	Minimax	1	['(3,6)', '(3,	(3,2)	-85000	15	1.9	ĐẠT
T03	Minimax	2	['(3,6)', '(3,	(3,2)	-120000	245	32.1	ĐẠT
T03	Minimax	3	['(3,6)', '(3,	(3,2)	-100000	4080	540.6	ĐẠT
T03	Minimax	4	['(3,6)', '(3,	(3,2)	-100000	83102	10756	TRƯỢT
T03	Alpha-Beta	1	['(3,6)', '(3,	(3,2)	-85000	15	2.1	ĐẠT
T03	Alpha-Beta	2	['(3,6)', '(3,	(3,2)	-120000	103	12.6	ĐẠT
T03	Alpha-Beta	3	['(3,6)', '(3,	(3,2)	-100000	1336	188.8	ĐẠT
T03	Alpha-Beta	4	['(3,6)', '(3,	(3,2)	-100000	7345	1002.6	TRƯỢT

[T04] Chặn Thắng Hàng Dọc

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T04	Minimax	1	['(0,4)']	(0,4)	0	15	1.8	ĐẠT
T04	Minimax	2	['(0,4)']	(0,4)	-40000	238	29.3	ĐẠT
T04	Minimax	3	['(0,4)']	(0,4)	-30000	4005	522.6	ĐẠT
T04	Minimax	4	['(0,4)']	(0,4)	-195000	78203	10091	ĐẠT
T04	Alpha-Beta	1	['(0,4)']	(0,4)	0	15	1.9	ĐẠT
T04	Alpha-Beta	2	['(0,4)']	(0,4)	-40000	55	7.4	ĐẠT
T04	Alpha-Beta	3	['(0,4)']	(0,4)	-30000	389	51.4	ĐẠT
T04	Alpha-Beta	4	['(0,4)']	(0,4)	-195000	7105	859.8	ĐẠT

[T05] Đánh Đổi Tấn Công vs Phòng Thủ

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T05	Minimax	1	['(2,6)', '(2,	(2,2)	100000	15	1.7	ĐẠT
T05	Minimax	2	['(2,6)', '(2,	(2,2)	100000	215	27.9	ĐẠT
T05	Minimax	3	['(2,6)', '(2,	(2,2)	100000	3568	435.7	ĐẠT
T05	Minimax	4	['(2,6)', '(2,	(2,2)	100000	66379	8544.8	ĐẠT
T05	Alpha-Beta	1	['(2,6)', '(2,	(2,2)	100000	15	1.8	ĐẠT
T05	Alpha-Beta	2	['(2,6)', '(2,	(2,2)	100000	98	11.6	ĐẠT
T05	Alpha-Beta	3	['(2,6)', '(2,	(2,2)	100000	787	97.3	ĐẠT
T05	Alpha-Beta	4	['(2,6)', '(2,	(2,2)	100000	3811	481.4	ĐẠT

[T06] Tạo Nhánh – Mối Đe Dọa Kép

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T06	Minimax	3	['(5,2)', '(2,	(2,5)	100000	3045	378.1	TRƯỢT
T06	Minimax	4	['(5,2)', '(2,	(2,5)	100000	55503	7221.5	TRƯỢT
T06	Alpha-Beta	3	['(5,2)', '(2,	(2,5)	100000	357	41.6	TRƯỢT
T06	Alpha-Beta	4	['(5,2)', '(2,	(2,5)	100000	1723	216.5	TRƯỢT

[T07] Chặn Nhánh Của Đối Thủ

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T07	Minimax	2	['(5,2)', '(2,	(2,5)	-85000	185	24.1	ĐẠT
T07	Minimax	3	['(5,2)', '(2,	(2,5)	-25000	3045	400.1	ĐẠT
T07	Minimax	4	['(5,2)', '(2,	(2,5)	-195000	56791	7374.4	TRƯỢT
T07	Alpha-Beta	2	['(5,2)', '(2,	(2,5)	-85000	79	9.8	ĐẠT
T07	Alpha-Beta	3	['(5,2)', '(2,	(2,5)	-25000	574	71.9	ĐẠT
T07	Alpha-Beta	4	['(5,2)', '(2,	(2,5)	-195000	6107	780.9	TRƯỢT

[T08] Phát Hiện Môi Đe Dọa Góc

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T08	Minimax	1	['(0,3)']	(0,3)	100000	7	0.7	ĐẠT
T08	Minimax	2	['(0,3)']	(0,3)	100000	53	6.6	ĐẠT
T08	Minimax	3	['(0,3)']	(0,3)	100000	605	72.2	ĐẠT
T08	Alpha-Beta	1	['(0,3)']	(0,3)	100000	7	0.8	ĐẠT
T08	Alpha-Beta	2	['(0,3)']	(0,3)	100000	12	1.3	ĐẠT
T08	Alpha-Beta	3	['(0,3)']	(0,3)	100000	61	7.6	ĐẠT

[T09] Thăng Hàng Cuối

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T09	Minimax	1	['(8,6)', '(8,	(8,2)	100000	9	0.9	ĐẠT
T09	Minimax	2	['(8,6)', '(8,	(8,2)	100000	78	9.5	ĐẠT
T09	Minimax	3	['(8,6)', '(8,	(4,4)	100000	1056	120.5	TRƯỢT
T09	Alpha-Beta	1	['(8,6)', '(8,	(8,2)	100000	9	0.9	ĐẠT
T09	Alpha-Beta	2	['(8,6)', '(8,	(8,2)	100000	72	8.9	ĐẠT
T09	Alpha-Beta	3	['(8,6)', '(8,	(4,4)	100000	369	44.2	TRƯỢT

[T10] Vị Trí Đối Xứng – Kiểm Soát Trung Tâm

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T10	Minimax	1	['(3,3)', '(3,	(3,3)	0	9	1.1	ĐẠT
T10	Minimax	2	['(3,3)', '(3,	(3,3)	0	97	11.6	ĐẠT
T10	Alpha-Beta	1	['(3,3)', '(3,	(3,3)	0	9	1.1	ĐẠT
T10	Alpha-Beta	2	['(3,3)', '(3,	(3,3)	0	28	3.4	ĐẠT

[T11] Độ Nhạy Độ Sâu – Bẫy Phía Trước

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T11	Minimax	1	['(5,3)', '(5,	(1,3)	100000	19	2.0	ĐẠT
T11	Minimax	2	['(5,3)', '(5,	(1,3)	100000	303	42.1	ĐẠT
T11	Minimax	3	['(5,3)', '(5,	(1,1)	100000	6608	753.7	TRƯỢT
T11	Minimax	4	['(5,3)', '(5,	(1,1)	100000	124933	17725	TRƯỢT
T11	Alpha-Beta	1	['(5,3)', '(5,	(1,3)	100000	19	2.1	ĐẠT
T11	Alpha-Beta	2	['(5,3)', '(5,	(1,3)	100000	73	9.3	ĐẠT
T11	Alpha-Beta	3	['(5,3)', '(5,	(1,1)	100000	509	57.9	TRƯỢT
T11	Alpha-Beta	4	['(5,3)', '(5,	(1,1)	100000	2818	379.5	TRƯỢT

[T12] Hiệu Quả Alpha-Beta – Trung Cục Dày Đặc

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T12	Minimax	3	đo hiệu năng	None	100000	1	0.0	H.NĂNG
T12	Minimax	4	đo hiệu năng	None	100000	1	0.0	H.NĂNG
T12	Alpha-Beta	3	đo hiệu năng	None	100000	1	0.0	H.NĂNG
T12	Alpha-Beta	4	đo hiệu năng	None	100000	1	0.0	H.NĂNG

[T13] LỖI NGHIỆM TRỌNG – Không Chặn Chuỗi 3 Mở

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T13	Minimax	1	['(3,6)']	(3,6)	0	17	2.2	ĐẠT
T13	Minimax	2	['(3,6)']	(3,6)	-45000	311	38.5	ĐẠT
T13	Minimax	3	['(3,6)']	(3,6)	-25000	6035	797.9	ĐẠT
T13	Minimax	4	['(3,6)']	(2,2)	-100000	133034	18293	TRƯỢT
T13	Alpha-Beta	1	['(3,6)']	(3,6)	0	17	2.3	ĐẠT
T13	Alpha-Beta	2	['(3,6)']	(3,6)	-45000	109	14.1	ĐẠT
T13	Alpha-Beta	3	['(3,6)']	(3,6)	-25000	1756	230.1	ĐẠT
T13	Alpha-Beta	4	['(3,6)']	(2,2)	-100000	5288	688.2	TRƯỢT

[T14] LỖI NGHIỆM TRỌNG – Lỗi Loại Trừ score_window

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T14	Minimax	1	['(4,6)', '(4,	(3,7)	0	17	2.2	TRƯỢT
T14	Minimax	2	['(4,6)', '(4,	(3,7)	-35000	309	39.2	TRƯỢT
T14	Minimax	3	['(4,6)', '(4,	(3,7)	-20000	5946	784.6	TRƯỢT
T14	Minimax	4	['(4,6)', '(4,	(2,2)	-100000	128134	17634	TRƯỢT
T14	Alpha-Beta	1	['(4,6)', '(4,	(3,7)	0	17	2.3	TRƯỢT
T14	Alpha-Beta	2	['(4,6)', '(4,	(3,7)	-35000	107	14.0	TRƯỢT
T14	Alpha-Beta	3	['(4,6)', '(4,	(3,7)	-20000	1785	233.4	TRƯỢT
T14	Alpha-Beta	4	['(4,6)', '(4,	(2,2)	-100000	4839	632.0	TRƯỢT

[T15] Cuối Cược – Gàn Đầy Bàn

Mã	Thuật toán	Độ sâu	Nước kỳ vọng	Nước thực tế	Điểm số	Trạng thái	T.gian (ms)	Kết quả
T15	Minimax	1	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT
T15	Minimax	2	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT
T15	Minimax	3	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT
T15	Minimax	4	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT
T15	Alpha-Beta	1	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT
T15	Alpha-Beta	2	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT
T15	Alpha-Beta	3	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT
T15	Alpha-Beta	4	['(2,4)', '(6,	None	100000	1	0.0	TRƯỢT

3. Nhận xét về số trạng thái đã xét và thời gian chạy

- So sánh MINIMAX vs ALPHA-BETA

Mã	Độ sâu	TT Minimax	TT Alpha-Beta	Tỷ lệ A-B/MM	MM T.gian(ms)	A-B T.gian(ms)
T01	1	15	15	1.000	1.9	1.9
T01	2	215	98	0.456	26.4	12.1
T01	3	3,613	832	0.230	448.2	101.6
T01	4	68,988	4,180	0.061	8851.1	551.1
T02	1	17	17	1.000	1.9	1.9
T02	2	253	31	0.123	32.2	3.6
T02	3	4,475	253	0.057	519.3	30.4
T02	4	76,375	461	0.006	10236.6	56.3

T03	1	15	15	1.000	1.9	2.1
T03	2	245	103	0.420	32.1	12.6
T03	3	4,080	1,336	0.327	540.6	188.8
T03	4	83,102	7,345	0.088	10756.1	1002.6
T04	1	15	15	1.000	1.8	1.9
T04	2	238	55	0.231	29.3	7.4
T04	3	4,005	389	0.097	522.6	51.4
T04	4	78,203	7,105	0.091	10091.9	859.8
T05	1	15	15	1.000	1.7	1.8
T05	2	215	98	0.456	27.9	11.6
T05	3	3,568	787	0.221	435.7	97.3
T05	4	66,379	3,811	0.057	8544.8	481.4
T06	3	3,045	357	0.117	378.1	41.6
T06	4	55,503	1,723	0.031	7221.5	216.
T07	2	185	79	0.427	24.1	9.8
T07	3	3,045	574	0.189	400.1	71.9
T07	4	56,791	6,107	0.108	7374.4	780.9
T08	1	7	7	1.000	0.7	0.8
T08	2	53	12	0.226	6.6	1.3
T08	3	605	61	0.101	72.2	7.6
T09	1	9	9	1.000	0.9	0.9
T09	2	78	72	0.923	9.5	8.9
T09	3	1,056	369	0.349	120.5	44.2
T10	1	9	9	1.000	1.1	1.1
T10	2	97	28	0.289	11.6	3.4
T11	1	19	19	1.000	2.0	2.1
T11	2	303	73	0.241	42.1	9.3
T11	3	6,608	509	0.077	753.7	57.9
T11	4	124,933	2,818	0.023	17725.3	379.5
T12	3	1	1	1.000	0.0	0.0
T12	4	1	1	1.000	0.0	0.0
T13	1	17	17	1.000	2.2	2.3
T13	2	311	109	0.350	38.5	14.1
T13	3	6,035	1,756	0.291	797.9	230.1
T13	4	133,034	5,288	0.040	18293.4	688.2
T14	1	17	17	1.000	2.2	2.3
T14	2	309	107	0.346	39.2	14.0
T14	3	5,946	1,785	0.300	784.6	233.4
T14	4	128,134	4,839	0.038	17634.8	632.0
T15	1	1	1	1.000	0.0	0.0
T15	2	1	1	1.000	0.0	0.0
T15	3	1	1	1.000	0.0	0.0
T15	4	1	1	1.000	0.0	0.0

Từ dữ liệu thực nghiệm trên toàn bộ 15 kịch bản, ta có các kết luận mang tính định lượng rõ rệt:

- Tính toán vận của cắt nhánh (0 Phân kỳ): Trên tổng số hàng trăm nghìn trạng thái được duyệt, Alpha-Beta và Minimax không có bất kỳ sự phân kỳ (khác biệt) nào về nước đi. Điều này chứng minh logic điều kiện $\beta \leq \alpha$ được cài đặt

hoàn toàn chuẩn xác, không vô tình cắt bỏ các nhánh chứa nước đi tối ưu.

- Hiệu năng cắt giảm cực lớn: Tổng số trạng thái Minimax phải xét trên toàn bộ bài test là 920,186 trạng thái, trong khi Alpha-Beta chỉ cần xét 53,711 trạng thái. Trung bình, Alpha-Beta giúp cắt giảm 94.2% khối lượng công việc.

- Chênh lệch thời gian tại độ sâu cao: Sự chênh lệch này thể hiện rõ nhất ở độ sâu $d=4$. Trung bình Minimax phải xét tới 79,222 trạng thái và tiêu tốn 10,611.8 ms (~ 10.6 giây) cho một nước đi. Ngược lại, Alpha-Beta ép con số này xuống chỉ còn 3,971 trạng thái, thời gian chạy rút gọn còn 513.5 ms (giảm 95.0% thời gian).

4. Nhận xét về ảnh hưởng độ sâu tìm kiếm

- Do hiện tượng bùng nổ tổ hợp của thuật toán tìm kiếm trên không gian trạng thái, thời gian chạy tăng theo hàm mũ khi tăng độ sâu. Theo số liệu ghi nhận của Minimax: độ sâu 1 mất ~ 1.5 ms, độ sâu 2 mất 24.6 ms, độ sâu 3 mất 444.1 ms, nhưng lên độ sâu 4 thời gian vọt lên tới 10,611.8 ms

- Độ sâu tỷ lệ thuận với "tầm nhìn chiến thuật" của AI:

+ Độ sâu thấp ($d=1, d=2$): AI có lối chơi "tham lam", chỉ nhìn thấy lợi ích hoặc nguy hiểm ngay trước mắt. AI đủ sức xử lý các nước đi cơ bản như ăn ngay nếu có 3 quân, hoặc chặn ngay nếu đối thủ có 3 quân.

+ Độ sâu cao ($d \geq 3$): Chất lượng nước đi tăng vọt, AI bắt đầu biết "giăng bẫy". Nhờ nhìn xa hơn, AI có thể phát hiện các chuỗi nước đi thắng nhiều bước, đặc biệt là nhận diện và tung ra chiến thuật "Tạo nhánh" (Mồi đe dọa kép) – điều mà độ sâu 1-2 hoàn toàn bất lực.

V. TỔNG KẾT

1. Ưu điểm và hạn chế của chương trình

- Ưu điểm: Hàm đánh giá trạng thái sử dụng kỹ thuật cửa sổ trượt kết hợp hệ thống trọng số ưu tiên phòng thủ cực cao (Phạt -80,000\$ khi đối thủ có chuỗi 3). Điều này giúp AI nhạy bén, phát hiện và dập tắt ngay lập tức các đợt tấn công 1-ply của người chơi ở mọi hướng (ngang, dọc, chéo).

- Hạn chế:

- + Mất cân bằng trọng số: Điểm chặn là \$-80,000\$ nhưng điểm thưởng tấn công (chuỗi 3 của AI) chỉ là \$+5,000\$\$. Điều này khiến AI chơi quá an toàn, đôi khi mãi mê chặn đối thủ mà bỏ lỡ đường thắng của chính mình.

- + Lỗi đếm đôi và bù trừ: Ở các cửa sổ hỗn hợp chứa cả X và O, hệ thống không cộng dồn điểm chính xác mà trả về 0. Đồng thời, kỹ thuật cửa sổ 4 ô đôi khi đếm trùng lặp các mối đe dọa chồng lên nhau, làm sai lệch tổng điểm.

- Trường hợp chơi tốt: AI xử lý hoàn hảo trong các tình huống khai cuộc, trung cuộc đơn giản, hoặc khi có đe dọa rõ ràng trên 1 đường thẳng (Các kịch bản T01 đến T05, T08, T09).

- Trường hợp chưa hợp lý:

- + Bị "mù" trước chuỗi 3 mở hai đầu (.XXX.) do hàm đánh giá không phân biệt được chuỗi mở hai đầu và chuỗi bị chặn một đầu (T13).

- + Bỏ sót đường thắng của chính mình nếu chuỗi thắng đó nằm đè lên một cửa sổ có chứa 1 quân của đối phương (Lỗi loại trừ score_window - T14).

- Dựa trên thực trạng của chương trình, nếu tiếp tục phát triển, nhóm sẽ ưu tiên 3 cải tiến cốt lõi sau:

- + Cải tiến Hàm đánh giá: Tách biệt việc chấm điểm quân Máy và quân Người để tránh xung đột cửa sổ hỗn hợp. Bổ sung việc nhận diện trực tiếp chuỗi "3 mở hai đầu" (.XXX.) với mức phạt cực nặng (-120,000\$ thay vì chỉ dùng cửa sổ 4 ô.

- + Sắp xếp nước đi: Trước khi đưa danh sách nước đi vào vòng lặp Alpha-Beta, nhóm sẽ dùng hàm Heuristic để sắp xếp các nước đi tiềm năng nhất lên đầu. Điều này sẽ giúp Alpha-Beta cắt nhánh sớm hơn và nhiều hơn, đưa tỷ lệ tối ưu tiệm cận mức lý tưởng.

- + Tích hợp Bảng chuyển vị: Lưu trữ kết quả của các trạng thái bàn cờ đã xét vào bộ nhớ đệm (Cache) để tái sử dụng, tránh việc AI phải lặp lại phép tính cho những thế cờ trùng lặp. Cải tiến này hứa hẹn tăng tốc độ tính toán lên gấp 3 đến 5 lần.

2. Link repository Github của nhóm

https://github.com/manhngoc2682005-jpg/23021316_23021370

3. Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu

- Tài liệu môn học: Slide bài giảng môn Trí tuệ Nhân tạo
- Sách tham khảo nền tảng: Artificial Intelligence: A Modern Approach (Stuart Russell & Peter Norvig) - Chương 5: Adversarial Search
- Tài liệu lập trình:
 - + Thư viện tkinter: Xây dựng giao diện đồ họa.
 - + Thư viện time, copy: Đo lường hiệu năng và xử lý ma trận trạng thái.
- Phần đã tham khảo từ các Repository / Tutorial mẫu:
 - + Nguồn tham khảo: Repository Connect4-Python của lập trình viên / nhà nghiên cứu Keith Galli (Link: <https://github.com/KeithGalli/Connect4-Python>).
 - + Phần kế thừa: Kế thừa ý tưởng cốt lõi về việc sử dụng Kỹ thuật Cửa sổ trượt (Sliding Window) có kích thước 4 ô để thiết kế hàm đánh giá (Heuristic Evaluation). Đồng thời, nhóm cũng áp dụng triết lý gán trọng số bất đối xứng (Asymmetric Weighting) từ mã nguồn này, trong đó đặt mức điểm phạt cực kỳ nặng khi đối phương hình thành chuỗi 3 quân nhằm kích hoạt cơ chế phòng thủ tức thời của AI.